Pacing Gain and Congestion-Window Growth: Packet Models and SCP Experiments

Skai Uijing
Independent Researcher
China
skaiuijing@gmail.com

Abstract

Adding pacing to SCP raised two questions: how does gain affect window growth, and how far apart are a feedback generation's first and last packets? We develop the answer from continuous window models to a whole-packet next-slot scheduler, following a packet, its returning ACK and the last new packet that ACK permits. The author's integral-sandwich construction connects the layers. In slow start, summing discrete gaps gives limiting span $2\ln 2/g$ and window multiplier $\min(e^{g/2}, 2)$. In congestion avoidance, ordinary replacements preserve ACK phase while extra whole-packet credit moves through a group. A quadratic ACK-count law gives a birth-front advance of $3/2$ positions per growing group and a size cycle $q \to 3q$; an integral sandwich gives front work $3\ln 3/(2g)$. This motivates the fixed engineering choice $g_{\mathrm{CA}} = 3\ln 3/2$, replacing an earlier continuous adaptive reference. Clock-driven Python gain sweeps show a near-RTT span plateau and compression at larger gain. With the selected CA constant, late groups span 99.569 ms in both Python and actual SCP under 100 ms feedback after isolating clock quantization and per-ACK byte truncation. A microsecond clock and retained CA division remainder remove the earlier 88 ms native plateau. The resulting engineering policy uses fixed phase-specific gains, with the packet construction and execution evidence presented together.

CCS Concepts

• **Networks** → **Transport protocols**.

Keywords

pacing, slow start, congestion avoidance, congestion window, virtual-network experiments

1 Introduction

The implementation question. While building SCP, I replaced burst sending with pacing and observed slower transfers without a comparable change in the useful-data share. This was the starting question, not a controlled result from the campaigns reported below. I wanted to understand what changed in the feedback loop, and how to spread packets without unnecessarily slowing window growth.

A packet-sized construction. Send one packet at time zero. Its ACK returns at R and increases the window to two packets. If the next packets leave at R and $1.5R$, only one has been acknowledged at $2R$: the window is three, not four. This example made departure times central to the question. Immediate sending and a fixed-batch comparator then supplied the sandwich comparisons in the discrete comparison below; comparing their packet times matters more than guessing an entire window sequence.

Following the last packet. I proposed tracing a group's final packet and then the last packet permitted by its ACK. The invariant is small: a packet returns an ACK one feedback RTT after departure; in slow start that ACK releases one occupied position and adds one new position, permitting two packets. Repeating the construction turns the span question into accumulated gaps. Formalizing it revealed that the ACK can also wait behind old reservations and that the group's first packet can drift. The discrete SS construction retains these terms and shows how the tail construction leads to the limiting span.

Why CA reopened the question. In congestion avoidance, I expected a larger gain to compress sending when successive windows were nearly equal. Similar measured spans across different gains therefore prompted another inspection of the packets and their scheduling history. A CA ACK normally replaces only one packet; an extra packet appears when accumulated growth crosses a whole-packet threshold. I proposed scanning initial windows and gain, then working backward from the numerical boundary to a formula. Values near 1.648 led to the candidate $3\ln 3/2$ and to following the position at which an extra whole packet appears. The earlier continuous adaptive-gain calculation remains a useful comparison. The early scheduling rule and the current next-slot rule must be distinguished. Later SCP comparisons also exposed integer-byte growth effects. These were successive questions and corrections, not premises known at the outset.

Rebuilding the connection. Comparing whole groups exposed a mistake in the earlier interpretation: a tail past its nominal RTT boundary is not the same as an actual first-to-last span exceeding an RTT, because the group start can move. We retain that correction rather than treating the earlier numerical boundary as a theorem for a different scheduler. Section 5 follows the birth front and keeps actual span, the retained trailing slot and group-start delay separate.

The proof route. We keep the author's sandwich method visible: count the permitted packets, bound their gaps using the window formula, sum them and squeeze the sum between integrals. The continuous formula is an estimation tool inside a discrete argument. Continuous-rate and full-flight models are also retained as models in their own right, with their different assumptions explicit. Each sum is identified as service work, actual span or front work before connecting it to packet timing.

Making the reasoning executable. We follow the construction through mathematics, an independent Python packet model, and the actual SCP core in a virtual network. Actual departure starts the fixed feedback delay; both simulations advance their

clocks tick by tick. A separate ACK-index evaluator checks the model's output; it is not counted as network or protocol execution. Each layer has its own evidentiary role, and arithmetic, event-order and observation-phase differences remain explicit.

2 Continuous Window References

We first derive tractable growth references, then use continuous leading terms inside discrete gap sums. Time is normalized by the complete feedback RTT R, and window by MSS M. The continuously updated-rate and full-flight assumptions are explicit; neither replaces the packet queue studied next.

2.1 SS: a continuously updated rate

The continuous model updates the entire target rate with the current window, whereas the packet model preserves intervals already assigned under older windows. In normalized units its rate is $x(\tau) = gw(\tau)$ while credit remains available. Fixed feedback gives $w'(\tau) = x(\tau - 1) = gw(\tau - 1)$. Substitution of $w = Ae^{\lambda\tau}$ yields

$$\begin{aligned} \lambda e^{\lambda} &= g, \\ F(\tau) &= \int_{\tau-1}^{\tau} x(v)\,dv = (e^{\lambda} - 1)w(\tau). \end{aligned} \tag{1}$$

The flight constraint requires $e^{\lambda} \le 2$, reaching equality at $g = 2\ln 2$. Thus the two idealizations share a transition value but differ below it. At $g = 1$, the instantaneous-rate multiplier is approximately 1.763223, while the immutable-queue multiplier is $\sqrt{e} \approx 1.648721$. Previously reserved packets retain older, wider spacings in the latter model. An exponential ansatz also fixes a compatible history; it does not determine the amplitude generated by a single initial packet.

2.2 CA: derive the window before choosing gain

The author first asked for a CA window formula, then a gain depending on that window. To obtain a continuous full-window reference, allow continuously divisible packet amounts and a positive window $w(\tau)$, with normalized time $\tau = t/R$. Write $q(\tau)$ for actual sends per RTT. Fixed feedback and the CA increment give

$$w'(\tau) = \frac{q(\tau - 1)}{w(\tau)}. \tag{2}$$

The denominator is the window when feedback arrives, not when its packet was sent. In particular,

$$w_n - w_{n-1} = \int_{n-2}^{n-1} \frac{q(s)}{w(s+1)}\,ds. \tag{3}$$

This expresses directly how the preceding RTT's sends drive the next interval's growth.

The flight is $f(\tau) = \int_{\tau-1}^{\tau} q(s)\,ds$. Multiply (2) by w and integrate from τ to $\tau + 1$:

$$f(\tau) = \frac{w(\tau+1)^2 - w(\tau)^2}{2}. \tag{4}$$

The window constraint $f \le w$ therefore bounds $w(\tau + 1)$ by $\sqrt{w(\tau)^2 + 2w(\tau)}$. Equality throughout an interval requires

$$w(\tau + 1) = F(w(\tau)), \qquad F(w) = \sqrt{w^2 + 2w}. \tag{5}$$

At RTT boundaries this gives

$$\begin{aligned} w_{n+1} - w_n &= 1 - \frac{1}{2w_n} + O(w_n^{-2}), \\ w_n &= n - \frac{1}{2}\ln n + O(1). \end{aligned} \tag{6}$$

For the second line, the first gives $w_n = n + O(\ln n)$; substituting that estimate into the summed reciprocal correction gives the harmonic term. This is the continuous full-flight reference, not the integer SCP trajectory.

2.3 The continuous gain: first match the leading correction

On the full-flight branch, $f' = w'$. Combining $f' = q(\tau) - q(\tau - 1)$ with $q(\tau - 1) = ww'$ gives

$$q(\tau) = (w(\tau) + 1)w'(\tau). \tag{7}$$

For an autonomous target $q = g(w)w$, the time to grow from w to $F(w) = \sqrt{w^2 + 2w}$ must be one RTT. Hence

$$\int_{w}^{F(w)} \frac{1 + 1/u}{g(u)}\,du = 1. \tag{8}$$

Now use the continuous window formula inside this integral, retaining the order needed for the gain correction. Write $g(w) = 1 + A/w + O(w^{-2})$. Since $F(w) - w = 1 - 1/(2w) + O(w^{-2})$,

$$\begin{aligned} 1 &= \int_{w}^{F(w)} \left(1 + \frac{1 - A}{u} + O(u^{-2})\right) du \\ &= 1 + \frac{1/2 - A}{w} + O(w^{-2}). \end{aligned} \tag{9}$$

Matching the coefficient gives

$$\boxed{g(w) = 1 + \frac{1}{2w} + O(w^{-2}).} \tag{10}$$

This is first-order matching on a compatible continuous full-flight branch. It does not by itself prove existence of an exact gain or a packet cold-start guarantee.

In byte units, the simple first-order policy is $g_{\mathrm{CA}}(C) = 1 + M/(2C)$. Its maximum for $C \ge M$ is 1.5. This is the maximum of an approximation to a continuous adaptive gain; it does not establish the whole-packet fixed-gain value derived below. The large-window expansion supplies no small-window error guarantee at $C = M$.

There is a finite counterexample to zero drift under this policy. Apply the exact real-valued packet recurrence from $w = 1$ with $g(w) = 1 + 1/(2w)$, so the normalized gap is $1/(w + 1/2)$. Exact rational arithmetic through ACK 12 gives five packets in group 5, with

$$\begin{aligned} L_5/R &\approx 0.877778113 < 1, \\ \Omega_5/R &\approx 1.053289641 > 1. \end{aligned} \tag{11}$$

The inequalities are checked on rational values, not rounded decimals. The next start is delayed by approximately $0.053289641R$. Thus the continuous construction remains valid on its compatible history, but this first-order packet policy does not guarantee zero drift from the specified cold start.

This short calculation remains an important step in the research: it accounts for growing continuous flight, but it does not represent whole-packet birth and inherited ACK phase. The packet counterexample explains why the final engineering policy below is fixed. A continuously full-flight reference also needs compatible prehistory; it is not the one-packet empty-network initial state used in the experiments.

3 ACK accounting and two comparison models

Assume a fixed send-to-ACK delay R, no loss, continuously available data, one ACK per full-sized packet, and indefinite slow start. Mathematical packet times are initially exact real numbers; packets remain full MSS units. Clock quantization is introduced in the execution section. Each ACK adds one MSS, denoted M, to the window. Normalize time and window as $\tau = t/R$ and $w = \text{cwnd}/M$. With actual packet-send times s_j in normalized units, including $s_0 = 0$,

$$\begin{aligned} w(\tau) &= 1 + \sum_{j\ge 0} \mathbf{1}_{\{s_j+1\le\tau\}}, \\ W_{n+2} &= W_{n+1} + S_n. \end{aligned} \tag{12}$$

Here $W_n = w(n)$ is sampled after all boundary events, and S_n counts sends in $(n, n+1]$. Sampling at RTT boundaries does not defer ACK processing.

Immediate sending. ACKs in $(n, n+1]$ can only acknowledge packets already in flight just after n. If their number is $\mathcal{F}_n$, then $W_{n+1} - W_n \le \mathcal{F}_n \le W_n$. Thus $W_n \le 2^n$. Sending both newly permitted packets immediately at every ACK attains $\text{cwnd}(nR) = M2^n$ under these ideal assumptions.

Fixed batches. A separate comparator reads its window V_n at each boundary and sends at $n + i/V_n$, for $i = 0, \dots, V_n - 1$. ACKs update the window immediately but leave that batch unchanged. The next boundary supplies the next batch's first packet, so $S_n = V_n$ and

$$\begin{aligned} V_{n+2} &= V_{n+1} + V_n, \quad V_0 = 1,\ V_1 = 2, \\ \text{cwnd}_{\text{fixed}}(nR) &= MF_{n+2} = \frac{M(\phi^{n+2} - \psi^{n+2})}{\sqrt{5}}. \end{aligned} \tag{13}$$

where $\phi = (1+\sqrt{5})/2$, $\psi = (1-\sqrt{5})/2$, and $F_0 = 0, F_1 = 1$. This gives $1, 2, 3, 5, 8, 13, \dots$. It is a deliberately simplified comparator. For any scheduler satisfying the additional service condition $S_n \ge W_n$, induction in (12) gives $MF_{n+2} \le \text{cwnd}(nR) \le M2^n$. The lower condition requires justification for the scheduler being studied; earlier reservation alone is not a proof. Appendix D proves the lower comparison for the specific queued reference below by comparing actual packet times. It is not assumed for every pacer. These bounds motivate exponential growth but do not establish an exact exponential solution.

4 SS: Follow the Tail, Sum the Gaps, and Squeeze

4.1 The packet and its returning ACK

One packet leaves at zero. Its ACK returns at one RTT, releases one occupied position and increases the window by one MSS. Exactly two full packets become eligible. Each reservation uses the updated window, and the sender retains the interval after the last packet. In normalized time the rule is

$$\begin{aligned} a_k &= s_{k-1} + 1, & b_k &= \max(a_k, T_{k-1}), \\ s_{2k-1} &= b_k, & s_{2k} &= b_k + \frac{1}{g(k+1)}, \\ T_k &= b_k + \frac{2}{g(k+1)}. \end{aligned} \tag{14}$$

Here s_j is packet j's departure, a_k its returning ACK time, b_k the first new reservation, and T_k the next available slot. Initialize $s_0 = 0, T_0 = 1/g$. Reservations occupy window credit before departure: actual flight plus reserved backlog equals the window after each ACK.

A small example before the sums. Set $g = 2$. The first ACK reserves packets at 1 and 5/4. The ACK of the packet at 1 arrives at 2, raises the window to three and reserves packets at 2 and 13/6. Their trailing slot is 7/3. The next ACK arrives at $9/4 < 7/3$, raises the window to four, and reserves at 7/3 and 59/24. Thus even the last parent's ACK can find earlier reservations still present. These are ideal fractional times, not timestamps from the millisecond SCP clock.

4.2 Which gaps belong to the tail span?

Generations are defined by parent packets: the initial packet is generation zero, and its ACK's children form generation one. A generation of $W = 2^m$ parents produces $2W$ children. The corresponding ACKs have indices $W, \dots, 2W-1$, so their post-ACK windows are $W+1, \dots, 2W$.

For this scheduler and fixed $g \ge 1$, reservations within the child group are continuous. The check is that an internal ACK never arrives after the preceding reservation slot has expired. Appendix A checks this inequality from (14), using the monotonicity of $D(W) = \sum_{k=W+1}^{2W} 1/k$. The example above shows the queue wait that this check must retain.

Each ACK adds two intervals at its new window. Consequently the total service work is $2R\sum_{k=W+1}^{2W} 1/(gk)$. It includes the final packet's trailing interval $R/(2gW)$; first-to-last span excludes that interval:

$$\frac{L_{m+1}}{R} = \frac{1}{g}\left(2\sum_{k=W+1}^{2W}\frac{1}{k} - \frac{1}{2W}\right), \qquad W = 2^m. \tag{15}$$

For the four children in the example, this is $(1/3 + 1/4) - 1/8 = 11/24$, exactly $59/24 - 2$.

4.3 The integral sandwich

The continuous function $1/x$ supplies upper and lower bounds on the discrete intervals. Since it decreases,

$$\int_{W+1}^{2W+1}\frac{dx}{x} \le \sum_{k=W+1}^{2W}\frac{1}{k} \le \int_{W}^{2W}\frac{dx}{x}. \tag{16}$$

The two integrals tend to $\ln 2$, and the excluded tail interval tends to zero. Thus the author's tail construction gives

$$\boxed{\lim_{m\to\infty}\frac{L_{m+1}}{R} = \frac{2\ln 2}{g}}. \tag{17}$$

The integration estimates the discrete sum; it does not change the sender into continuously divisible packets.

4.4 From span to window growth

The same group uses $2D(W)/g$ RTTs of reservation service, including its tail interval. The next group waits for either the first returning ACK or that slot, whichever is later. Its start interval is therefore $\max(1, 2D(W)/g)$. This connects the sending span to window growth. Since $D(W) \uparrow \ln 2$, the limiting transition is $g_c = 2\ln 2$. Below it, service eventually dominates and harmonic service time gives exponential window growth. At or above it, ACK availability limits the multiplier:

$$r(g) := \lim_{n\to\infty} \frac{W_{n+1}}{W_n} = \min(e^{g/2}, 2). \tag{18}$$

Here $W_n = w(n)$ is a time-boundary sample, distinct from a generation's parent count W. The same harmonic inversion gives the window trajectory:

$$\begin{aligned} w(\tau) &= A_g e^{g\tau/2} + O_g(1), && 1 \le g < g_c, \\ w(n+v) &= 2^{n-1}\min(e^{gv/2}, 2) + O_g(1), && g \ge g_c, \end{aligned} \tag{19}$$

where $n \ge 1$, $0 \le v < 1$, and A_g is fixed by the initial packet history, not fitted to the observations. At finite $g \ge g_c$, $W_n = 1 + 2^{n-1}$; the multiplier two does not remove the startup deficit relative to immediate sending. The exact event solution and tail-accounting check are in Appendix A. Higher gain compresses the group without raising this limiting multiplier above two.

5 CA: The Same Sandwich with Whole-Packet Credit

5.1 What one returning packet permits

The tail construction is unchanged: an actual departure generates an ACK one RTT later. What changes is the window update. In the real-valued CA form of RFC 5681, Section 3.1, equation (3) [1], ACK k gives $u_k = u_{k-1} + 1/u_{k-1}$. Window values and packet times are different quantities. From one MSS the post-ACK windows are $2, 5/2, 29/10$; those ACKs permit respectively two, one and one new full packets.

For the same $g = 2$ example used in SS, ACKs at times $1, 2, 9/4$ therefore reserve CA packets at $\{1, 5/4\}$, $\{2\}$ and $\{9/4\}$. The second child generation has span $1/4$, rather than the SS generation's $11/24$. The continuous window increase must cross an integer threshold before it creates a second full packet.

Initially reserve $p_0 = \lfloor a \rfloor$ packets, with $u_0 = a \ge 1$, starting at zero and leaving slot $T_0 = p_0/(ga)$. After ACK k,

$$\begin{aligned} m_k &= 1 + \lfloor u_k \rfloor - \lfloor u_{k-1} \rfloor, \\ b_k &= \max(a_k, T_{k-1}), \\ T_k &= b_k + \frac{m_k}{g u_k}. \end{aligned} \tag{20}$$

The ACK time a_k is the actual departure plus one RTT. The extra packet is admitted by the floor difference, not by rounding every group to an assumed one-MSS increase.

Why a smaller gap does not compress inherited phases. At $g = 3\ln 3/2$, the first two children leave at R and $R + R/(2g)$. Their ACKs return one RTT later with exactly the same separation. The first ACK sets the next slot to $2R + R/(2.5g)$, but the second ACK arrives later, at $2R + R/(2g)$. Its single replacement therefore retains the parent's phase. A smaller gap is permission to send earlier, not earlier availability of a window position. Old reservations can still delay a replacement; phase copying applies when its ACK finds the slot already available. This distinction explains why service work near R/g need not compress the whole group into R/g.

5.2 Use the continuous window formula to bound the discrete ACKs

First keep ACK count K separate from elapsed time. Treating ACK count continuously gives $du/dK = 1/u$, hence $u^2 = a^2 + 2K$. The discrete square identity tells us how far this approximation can be used:

$$\begin{aligned} u_{k+1}^2 - u_k^2 &= 2 + u_k^{-2}, \\ u_K^2 &= a^2 + 2K + \sum_{k=0}^{K-1} u_k^{-2} \\ &= a^2 + 2K + O(\ln(K+1)). \end{aligned} \tag{21}$$

The lower bound $u_k^2 \ge a^2 + 2k$ bounds the reciprocal sum above by a harmonic sum. Thus the continuous leading term has a controlled discrete error. If K_p is the first ACK index with $u \ge p$, threshold overshoot changes u^2 by only $O(1)$, so

$$K_p = \frac{p^2}{2} + O(\ln p). \tag{22}$$

This is enough for the front limit below. No elapsed-time formula $w_n = n - c\ln n + O(1)$ is assumed for this packet scheduler. The continuous full-flight time formula is derived separately later.

5.3 One group: squeeze its service work first

Suppose a parent group has $p = \lfloor w \rfloor$ packets and the window before its first ACK is w. Its p ACKs raise the window by at most $p/w \le 1$, and permit $N = p$ or $p + 1$ children. Each new interval lies between $R/[g(w+1)]$ and $R/(gw)$. Therefore their total service work S, including the trailing interval, obeys

$$\frac{N}{g(w+1)} \le \frac{S}{R} \le \frac{N}{gw}, \qquad \frac{S}{R} \longrightarrow \frac{1}{g}. \tag{23}$$

This sandwich counts assigned intervals. To recover actual first-to-last span, let I be the internal gaps spent waiting for later ACKs to make new data eligible, and let δ_{tail} be the last assigned interval. Exact-time reservation accounting gives

$$\Omega = S + I, \qquad L = S + I - \delta_{\text{tail}}. \tag{24}$$

In the CA example, service work is $1/5 + 5/29$, internal waiting is $1/20$, and the trailing gap is $5/29$; their combination is the calculated span $1/4$. Unlike the SS group, this CA group contains an ACK-limited idle interval. We must determine that waiting before identifying S with a tail span.

One-packet and fractional-window effects also matter when retaining a $1/w$ correction. From integer $p \ge 2$, exactly p ACKs give $u_p = p + \sum_{k=0}^{p-1} 1/u_k < p + 1$: that group has no extra whole packet. A claim that every group has $p + 1$ children would change the very order of correction being calculated.

5.4 Many groups: why a logarithm accumulates

To see what can be accumulated exactly, define the idle time of the reservation clock as

$$\mathcal{I}_K = \sum_{k=1}^{K} [a_k - T_{k-1}]_+.$$

This is waiting for new credit after the previous slot, distinct from the packet's queue wait η_k. Telescoping (20) gives

$$\begin{aligned} T_K &= T_0 + \mathcal{I}_K + \frac{A_K + B_K}{g}, \\ A_K &= \sum_{k=1}^{K} \frac{1}{u_k}, \qquad B_K = \sum_{k=1}^{K} \frac{\lfloor u_k \rfloor - \lfloor u_{k-1} \rfloor}{u_k}. \end{aligned} \tag{25}$$

The first sum pays for replacement packets; the second pays for the additional whole packets born from growth.

The replacement term has an exact finite sum. Shift the reciprocal sum by one ACK and use $1/u_{k-1} = u_k - u_{k-1}$:

$$\begin{aligned} A_K &= \sum_{k=1}^{K} \frac{1}{u_{k-1}} - \frac{1}{a} + \frac{1}{u_K} \\ &= u_K - a - \frac{1}{a} + \frac{1}{u_K}. \end{aligned} \tag{26}$$

For the extra-packet term, let v_n be the first post-ACK window satisfying $v_n \geq n$ for integer $n > p_0$. Its overshoot is at most $1/(n-1)$, giving

$$0 \leq \frac{1}{n} - \frac{1}{v_n} \leq \frac{1}{n^2(n-1)}.$$

The correction is summable. The crossed thresholds therefore contribute a harmonic sum plus a convergent correction, so

$$B_K = \ln u_K + C_B + O(1/u_K).$$

Consequently, for fixed $g \geq 1$ and indefinitely continued CA,

$$T_K = \mathcal{I}_K + \frac{u_K + \ln u_K}{g} + C + O(1/u_K). \tag{27}$$

This explains the linear contribution from replacements and the logarithmic contribution from occasional extra packets. The idle term has not disappeared: (27) is not a closed formula for $\mathrm{cwnd}(t)$, nor does it determine the limiting tail span by itself.

This is the multi-generation version of the same sandwich: replacement work grows with the window, and the extra whole packets contribute a harmonic sum. The two positive-part terms have different meanings: $[T - a]_+$ waits behind existing reservations, whereas $[a - T]_+$ waits for fresh ACK credit. Advancing an expired slot to now does not erase earlier group-start drift.

5.5 The extra packet moves from the group head to the tail

Follow where the next whole packet is born instead of predicting every departure. Choose the restart of a birth front: after a q-packet group produces no extra packet, the following q-packet group's first ACK crosses the next threshold. Its birth rank is $r_q = 1$. To check this, its starting window v satisfies $v^2 \geq q^2 + 2q$ and $v < q+1$, so $v + 1/v > q+1$. This restarts a credit rank, not the pacer's retained slot.

Until the next group without an extra packet, group sizes are $q, q+1, \ldots$. Let κ_p be the ACK count before processing the p-packet group. Then $r_p = K_{p+1} - \kappa_p$ and $\kappa_p - \kappa_q = \sum_{j=q}^{p-1} j$. Use the continuous leading term with its discrete error from (22):

$$\begin{aligned} r_p - 1 &= \frac{(p+1)^2 - (q+1)^2}{2} - \frac{(p-q)(p+q-1)}{2} + O(\ln q) \\ &= \frac{3}{2}(p-q) + O(\ln q). \end{aligned} \tag{28}$$

The estimate is uniform for $q \leq p \leq 4q$. The quadratic terms cancel; the remaining $3/2$ is the advance of the birth position, not the window increase per RTT.

Let b be the first group with $r_b > b$. It occurs before $4q$ for large q by the last formula. The first crossing overshoots the tail by at most $O(\ln q)$, so $r_b = b + O(\ln q)$. Substituting into (28) gives

$$b = 3q + O(\ln q), \qquad b/q \longrightarrow 3. \tag{29}$$

Thus the size interval $q \to 3q$ comes directly from moving the extra packet from the head to the tail.

Weight that front movement by the interval scale $1/(p+1)$:

$$\mathcal{W}_q = \sum_{p=q}^{b-1} \frac{r_{p+1} - r_p}{p+1}. \tag{30}$$

Put $e_p = r_p - 1 - 3(p-q)/2$, so $\max |e_p| = O(\ln q)$. A telescoping weighted sum bounds its entire contribution by

$$\left| \sum_{p=q}^{b-1} \frac{e_{p+1} - e_p}{p+1} \right| \leq \frac{2 \max |e_p|}{q+1} = O(\ln q/q).$$

The remaining harmonic sum has upper and lower integral bounds tending to $\ln(b/q) = \ln 3$. Therefore

$$\boxed{\mathcal{W}_q = \frac{3}{2} \ln 3 + O(\ln q/q) \longrightarrow \frac{3}{2} \ln 3.} \tag{31}$$

Clipping the final birth rank to the actual tail changes only $O(\ln q/q)$ here. This calculation needs the coarse controlled window approximation, not a high-order threshold expansion.

5.6 Return from the front sum to actual packet times

The front sum describes moving whole-packet credit. Connecting it to perfect pacing still requires the actual queue and group starts in (24). For a group with normalized first and last sends h_n, t_n, let θ_n be its final retained interval in RTT units. Then the next group starts at

$$\begin{aligned} h_{n+1} &= \max(h_n + 1, t_n + \theta_n), \\ E_n/R &= h_{n+1} - h_n - 1 = [L_n/R + \theta_n - 1]_+. \end{aligned} \tag{32}$$

Here $L_n = R(t_n - h_n)$, $\Omega_n = L_n + R\theta_n$, and cumulative start drift is $R(h_n - n)$. Thus $L_n < R$ can coexist with $E_n > 0$. Appendix B records the exact tail recursion, including old-queue waiting, an occasional birth gap and group-start motion.

For this exact-time real-window model, fixed $g \geq 1$ already guarantees every complete descendant group's actual span $L_n < R$; at $g = 1$ it tends to R. For $0 < g < 1$ it tends to $R/g > R$. Appendix C proves these claims. These elementary bounds explain why measured low-gain spans should be described as close to an RTT,

rather than forced into an inequality $L \geq R$ below the engineering CA value. The utilization question also includes the retained tail slot and the start delay in (32). We retain this distinction beside the operational constant rather than making an exact cold-start threshold the design criterion.

5.7 From the birth front to a fixed engineering policy

The three calculations now have distinct meanings. The one-group sum in (23) approaches $1/g$. Continuous full flight adds the first-order correction $1/(2w)$. The cross-generation whole-packet front instead gives $3\ln 3/(2g)$. Ordinary replacement ACKs can retain older phases, which a local service sum cannot compress. Setting the front-work reference to one RTT motivates our fixed choices:

$$\boxed{g_{\mathrm{SS}} = 2\ln 2, \qquad g_{\mathrm{CA}} = \tfrac{3}{2}\ln 3 \simeq 1.647918433.} \tag{33}$$

CA now uses this constant in both AIMD's CA phase and PROB. The latter has no separate slow-start gain. The earlier $1 + M/(2C)$ calculation remains a continuous reference and a historical control, not the current policy.

The CA choice is supported by the birth-front calculation and the engineering experiment below. Actual group span, occupied span and start delay are kept separate. At low gains the actual span can remain close to one RTT while starts move; greater gain compresses the group once this spreading plateau is passed.

6 From the Models to SCP Execution

6.1 Integer scheduling in the tested core

The experiment executes the SCP C sender, receiver, pacing callback, and ACK processing. The virtual network transports their packets; it does not compute the window from a recurrence. The core stores reservation times in unsigned 64-bit Q32.32 ticks. One tick is 1 ms by default; virtual builds use 1 μs through `SCP_TICK_SCALE=1000`. Each congestion controller supplies a Q16 gain K. AIMD selects $2\ln 2$ in SS and $3\ln 3/2$ in CA; PROB uses the same CA value throughout. For a fixed target g, payload length ℓ, measured RTT R_{tick}, and byte window C, scheduling uses

$$d_{\mathrm{Q32}} = \left\lfloor \frac{\ell R_{\mathrm{tick}} G}{C} \right\rfloor, \qquad K = \mathrm{round}(2^{16} g), \quad G = \left\lfloor 2^{48}/K \right\rfloor. \tag{34}$$

Write $g_Q = K/2^{16}$; reciprocal rounding adds the smaller difference between g_Q and $2^{32}/G$. At each output call the next slot becomes $\max(T, \mathrm{now})$. A packet receives that timestamp, then advances T by its own spacing. Fractional parts and the trailing slot survive an empty queue. This follows the next-slot principle in Linux v6.12 TCP [2]; SCP does not reproduce its initial-segment exemption, jitter compensation or qdisc/offload behavior. The send routine uses the reservation's integer tick part: on the old 1 ms clock, 1.8 ms was eligible at 1 ms; on a 1 μs clock, 1800.8 μs is eligible at 1800 μs. Both clocks advance tick by tick. The pacing timer now wakes for the queue head's due tick, rather than polling every tick; ACK-triggered scheduling shares the due-send routine.

The absence of SRTT uses a physical 1 ms fallback at either tick scale. Once feedback is available, the measured RTT is 100 ms. The rebuilt independent Python model uses the same next-slot rule and a 1 μs clock. Its default CA is real-valued and its original gain sweeps are unquantized; the controlled ablation below also implements both byte-update rules and uses the exact native Q16 effective gain. Native ticks drain forward arrivals, reverse arrivals, then timers. Python drains forward arrivals, reverse ACKs and sends; each ACK may itself send due packets. These are declared differences, not interchangeable implementations. Python sends only full MSS packets, totaling 1 GiB plus 846 bytes; native SCP sends exactly 1 GiB with a short final packet. The initial single-packet trailing slot uses known R in Python and the fallback in SCP; for the fixed gains $g \geq 1$ it expires before the first ACK in both.

Table 1: Frozen 24 October SCP virtual-network configuration. The later clock and byte-update ablation is reported separately.

Quantity	Configuration
Data and packet size	1 GiB generated data; MSS 1430 bytes; final payload 584 bytes
Window and mode	Initial window 1 MSS; separate sustained SS and CA campaigns
Network	50 ms per direction; no loss, rate cap, or serialization delay
Feedback RTT	100 ms from actual DATA send to returning ACK input
Observation	Core state every 1 ms; actual DATA and ACK events logged
Gains	$g_{\mathrm{c}} + 0.05k$, $k = -6, \ldots, 6$; 1, 1.1, 1.75, 2, 2.5, 3, 4, 6, 8
Repetitions	Per phase: 22 fixed gains × 3, plus native policy × 3
Limits	cwnd cap $2^{31} - 1$ bytes; buffer/window settings 1 GiB

6.2 Configuration and verification

In the frozen 24 October campaigns, all 138 native transfers completed with 750,869 DATA packets each and matching endpoint checksums. No loss, retransmission, or cwnd-cap contact occurred. Fifteen functional tests passed in each campaign. The shared core, library, virtual network, and common-transfer files exactly matched baseline `e41ac88`. CA is selected at the research entry by setting ssthresh to one MSS, without a core patch. Audits follow every DATA send through the receiver's input and ACK generation to the sender's ACK input, verify 100 ms feedback and every window update, and check actual sends against reserved timestamps. Three repetitions per configuration produced identical raw traces.

At target g_{c}, $K = 90852$, $g_Q = 1.38629150390625$, and $G = 3098170394$. The Q16 value is slightly below g_{c}, so the exact $g \geq g_{\mathrm{c}}$ theorem cannot simply be assigned to this finite run. The first DATA send defines time zero; the handshake is excluded. Repetitions are determinism checks, without random-error bars. Span summaries require complete, non-final-send intervals with at least two packets; all other intervals remain in the raw tables. Middle–late summaries use bins from 50% through 90% of the final-send bin index.

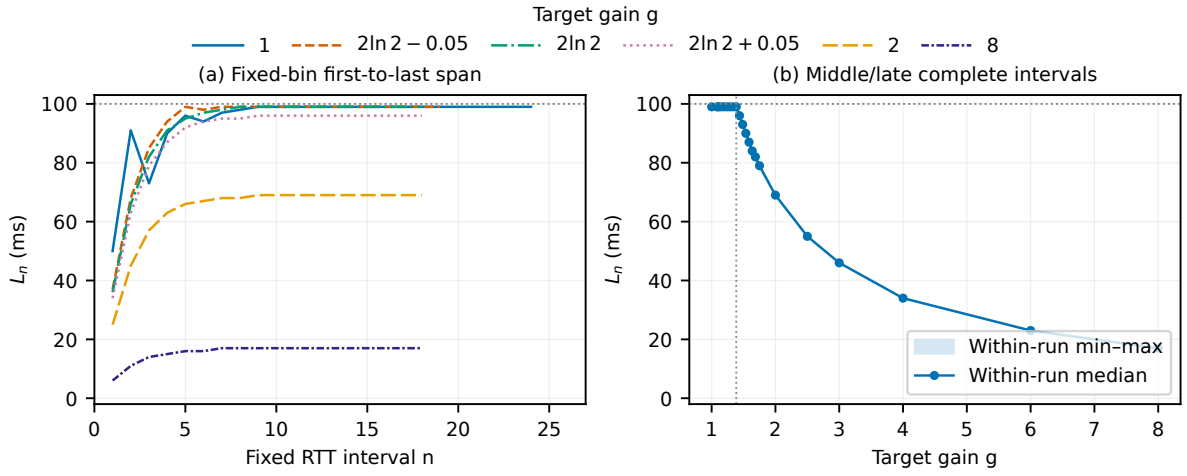


Figure 1: Actual SCP slow-start sending spans. Representative within-run curves are accompanied by the middle–late summary across all 22 gains. Incomplete, singleton, and final-send intervals are excluded from the summaries. Spans come from packet timestamps, not a nominal rate.

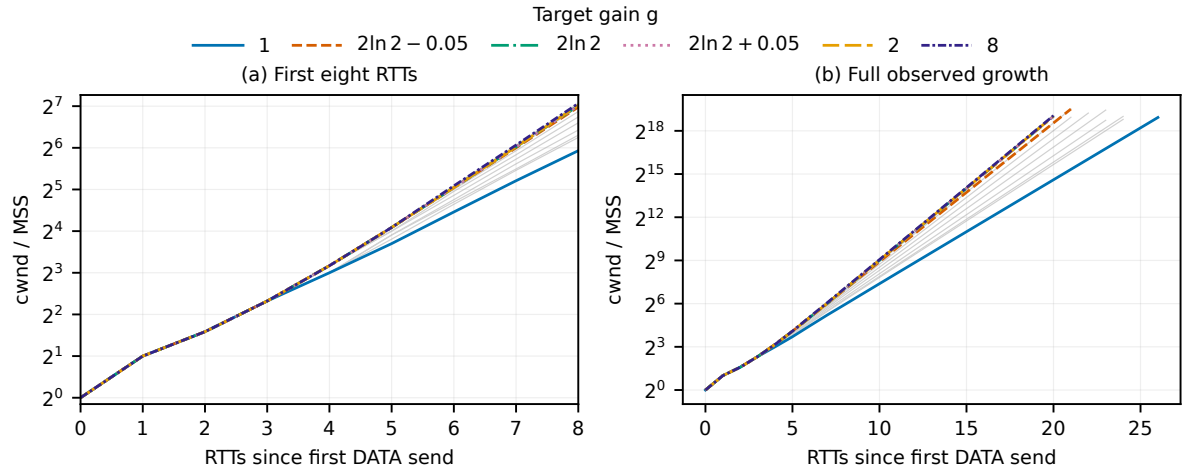


Figure 2: Actual SCP slow-start windows at RTT boundaries: all 22 gains in gray, six representative gains highlighted, with early and full-growth views. Time is measured from the first DATA send in units of the 100 ms feedback RTT; windows are in MSS on a base-two logarithmic axis. Lines connect samples.

7 Measured congestion-window growth

The low-gain regime. Figure 2 shows slower growth at low gain. For the eight sampled gains from 1 through $g_c - 0.05$, the measured boundary ratio W_{18}/W_{17} differs from $e^{gQ/2}$ by less than 0.003%. At $g = 1$ the ratio is 1.648756, close to the queued reference $\sqrt{e}$ and distinct from the instantaneous-rate prediction 1.763223. This supports the queued model's multiplier in this regime, without asserting equality of every timestamp or window value.

Doubling does not remove the startup deficit. At the critical target, boundary windows begin 1, 2, 3, 5, 9, 17, 33, 65, 129 MSS. Later values include $W_{14} = 8249$, $W_{18} = 131984$, and $W_{19} = 263968$.

Table 2: Measured growth and completion for selected gains.

Target gain	W_{18}/W_{17}	W_{18} (MSS)	Last ACK (t/R)
1	1.648756	9,144	26.82
$g_c - 0.10$	1.902456	73,216	21.62
$g_c - 0.05$	1.950624	100,818	21.01
g_c	2.000000	131,984	20.51
2	2.000000	132,390	20.35
8	2.000000	136,422	20.08

The samples double from RTT 14 through 19, yet W_{18} is 49.6521% below the immediate-send reference 2^{18}. At RTT 20 the window is 527935 MSS, one MSS below another exact doubling. The finite trace therefore supports a late multiplier near two without an exact infinite-time doubling or within-RTT periodicity claim.

8 Within-RTT sending span and concentration

For this figure, let $\mathcal{P}_n$ contain sends in the half-open interval $[nR, (n+1)R)$, measured from the first DATA transmission. Define

$$L_n^{\text{bin}} = \max_{j \in \mathcal{P}_n} t_j - \min_{j \in \mathcal{P}_n} t_j. \tag{35}$$

A singleton has span zero; an empty interval has no defined span. This bin convention differs from the right-closed send-count intervals in (12). It also differs from the packet-index groups defining L_m in (15). These measurements cannot be interchanged near a phase boundary.

Evolution within one run. At the critical target, L_n^{bin} begins 0, 36, 66, 82, 91, 95, 97, 98 ms for $n = 0, \ldots, 7$, then remains 99 ms for $n = 8, \ldots, 18$. The final-send interval has span 51 ms and is excluded from aggregates. The finite millisecond plateau is distinct from the harmonic convergence of the exact reference group's span.

Variation across gains. At RTT 18, fixed-bin spans are 99 ms at the critical target, 69 ms at $g = 2$, and 17 ms at $g = 8$. A 100 ms half-open interval sampled on a 1 ms grid has a maximum span of 99 ms. That value means sends reach both edge bins; it neither proves uniform density nor excludes a packet-index group extending across an RTT boundary. In particular, low-gain groups can exceed one RTT while their fixed-bin spans remain 99 ms.

A corrected implementation mismatch. The earlier Python script truncated each spacing to microseconds. At $w > 100000/g$ MSS the spacing became zero, so a finer execution clock did not preserve the intended reservation algorithm. At $g = 1$, that historical script completed at 25.21217 RTTs. The rebuilt script retains the fraction and completes at 26.82611 RTTs, against native SCP's 26.82. The old result is retained as evidence of the mismatch, not as a current next-slot comparison. Figure 3 uses the rebuilt script. A separate ACK-index evaluator checks its window, ACK count, reservation, group span and drift tables. Across 264 runs, all checked fields agree within the declared binary64 comparison tolerance; this consistency check is not an exact-arithmetic proof or protocol execution.

The three panels do not bound all gains' finite timing errors. Across the full rebuilt fixed-gain grid and policy controls, the largest common-boundary SS difference is 4.077% of the Python window, at $g = 8$, RTT 20. Both flows are still sending full-sized data there. This is an elapsed-time comparison, retaining the implementations' dispatch and gain differences, rather than a failure of the per-ACK window update.

From the critical target to $g = 8$, the last DATA ACK moves from 2051 to 2008 ms, an improvement of 2.10%. Meanwhile L_{18}^{bin} falls from 99 to 17 ms, and the largest millisecond bin's share of RTT-18 packets rises from 1.3813% to 7.7396%. Faster completion and more concentrated sending are distinct effects; span alone does not describe internal density.

Table 3: Python CA: late-group median span and accumulated delay at group 1200. Complete groups 900–1200, three identical repeats.

Gain	Median L (ms)	Start delay (ms)
1	99.964	320.165
1.5	99.89	32.538
$C - 0.05$	99.897	7.151
C	99.569	0.0
$C + 0.05$	96.637	0.0
2	82.026	0.0
4	40.972	0.0
8	20.465	0.0

9 CA Experiments: Spreading and Window Growth

9.1 A CA-centered Python gain sweep

The engineering question is measured within each run: does fixed $C = 3 \ln 3/2$ keep middle–late groups close to one feedback RTT, and does increasing gain shorten them? We ran the unchanged 1 μs clock model from one MSS in CA, with no loss and exactly 100 ms feedback. Each requested 1 GiB transfer is rounded up to full MSS packets. The 22 gains include 0.9, 1, 1.2, 1.4, 1.5, 1.9, 2, 2.5, 3, 4, 8 and $C + \{-.10, -.05, -.02, -.01, -.005, 0, .005, .01, .02, .05, .10\}$. All 66 runs completed; the three repetitions of every configuration have identical window and group CSV hashes. Deterministic repetitions are reported as repeatability checks, not statistical confidence intervals.

We prespecified groups 300–600 as middle and 900–1200 as late, excluding incomplete groups and the group containing the last packet. Every selected late group at C has $L = 99.569$ ms, within 0.431% of R, with no observed start drift. At $C + 0.005$, $C + 0.01$ and $C + 0.05$, spans fall to 99.282, 98.972 and 96.637 ms. Below C, spans lie near the RTT plateau while start delay can accumulate: at $C - 0.05$ the late span is 99.897 ms and accumulated delay at group 1200 is 7.151 ms. Gain 0.9 has late spans 110.984–111.040 ms. Thus the experiment supports a constant near C for the stated engineering objective, including the observed plateau and compression trend for the stated configuration.

The CA update at a matching full-packet ACK count is identical across these gains. At common elapsed RTT boundaries, the largest window difference from gain 1 in this sweep is 3.197881 MSS. Thus the spreading change is substantial while the observed CA window-time difference is small; the two research questions should not be collapsed into one plot.

9.2 Historical 1 ms fixed-constant SCP execution

We first ran the actual SCP core with the registered CA callback, not a model of that callback: three transfers of exactly 1 GiB, preceded by an 8 MiB path check. The local precommit research worktree records its source diff and hashes; it predates the clock and remainder changes below. The binary SHA-256 begins `af9ba7e5a274`; the complete identity is in `data/ca-native-constant.json`.

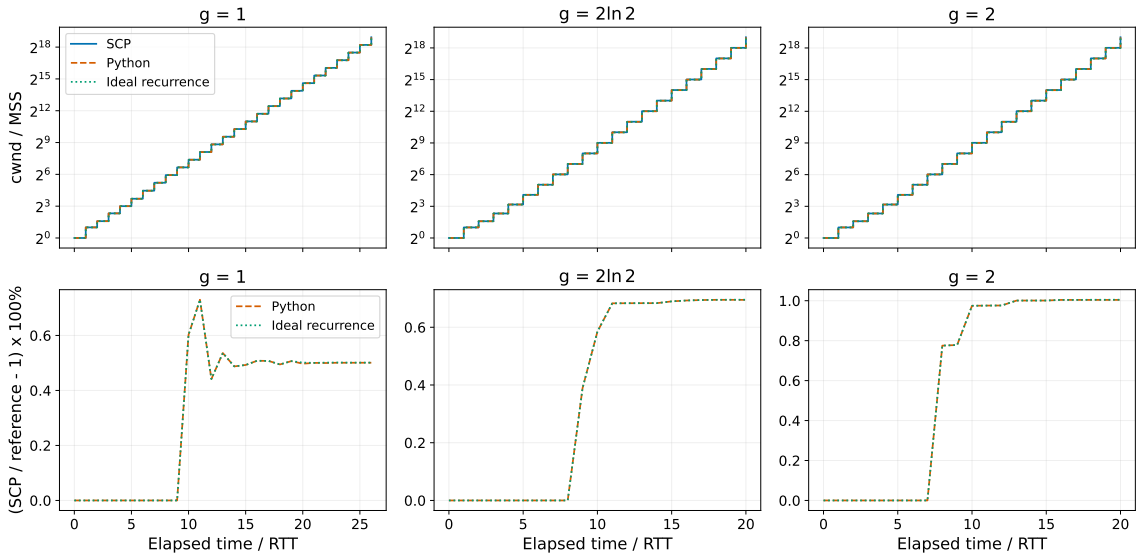


Figure 3: Rebuilt SS comparison at the same elapsed RTT boundaries. Top: observed SCP and clock-driven Python windows, plus continuous-time ACK-index recurrence evaluation. Bottom: native minus each reference, divided by that reference. No time shift or amplitude fit is applied. Fractional slots persist in both implementations; millisecond dispatch and gain quantization still shift ACK phase.

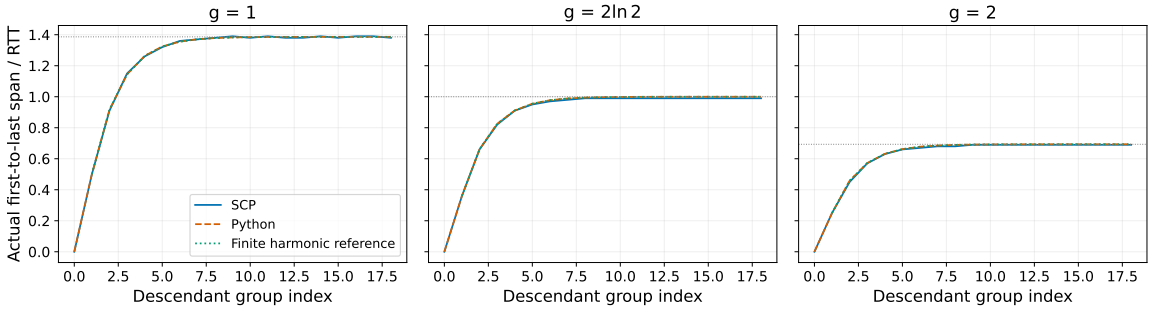


Figure 4: Complete, non-terminal descendant groups in observed SCP and rebuilt Python runs, against the finite harmonic reference (15) **at the unquantized target gain. Gray dotted lines show its limits. Native SCP uses the corresponding Q16 gain and millisecond dispatch. Unlike fixed-bin spans, a complete group's span can exceed one RTT.**

Each run sends 750,869 DATA packets, with a final 584-byte payload. Both endpoints obtain checksum `7a4bb4de42097f8a`.

Feedback is exactly 100 ms, with no loss or retransmission. Gain remains Q16 107998 throughout ($g_Q \simeq 1.647918701$). Each transfer completes in 1339.38 RTTs, and the three raw event-log hashes are identical. Reconstruction identifies 1,338 complete groups per run, with no unattributed packets or group-start rule errors.

In both selected ranges, groups 300–600 and 900–1200, actual SCP has $L = 88$ ms throughout and zero accumulated start delay. Late occupied span has median 88.732585 ms. Figure 8 places these observations beside Python's 99.569 ms at the unquantized target: the native span is 11.569 ms shorter. The implementations share the retained-slot principle, but native SCP has integer-byte CA growth, Q16 gain and a 1 ms clock, versus real windows, unquantized gain and 1 μs in Python. The observed difference is retained rather than fitted away; this experiment does not separate each arithmetic and clock contribution.

The final native window is 1,476,948 bytes, including the short final ACK; every update matches the integer law below with zero byte error. Thus the constant is exercised by the real transport path, while its 88 ms span also shows why an engineering gain selected from a model must still be checked in the target implementation.

9.3 Clock and CA-update ablation

The next experiment separates the two implementation effects instead of attributing the 88 ms plateau to the gain. All controls use fixed 100 ms feedback, no loss or serialization, one-MSS initial cwnd, persistent next-slot, and $g_Q = 107998/65536$. Changing only the native clock to 1 μs raises the late median span from 88

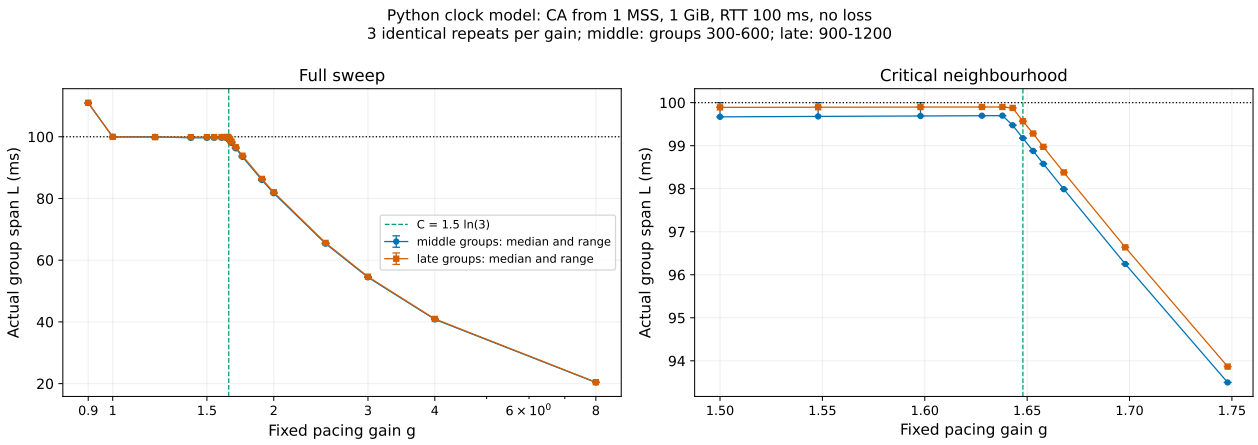


Figure 5: CA-centered Python clock sweep. Middle groups 300–600 and late groups 900–1200 are summarized by their median and within-run range. The right panel enlarges the neighborhood of $C = 3 \ln 3/2$. Three repeated runs per gain produce identical tables. The near-RTT plateau and greater-gain compression are observations, not values generated from the front formula.

Table 4: CA ablation at 1 μs, late complete groups. $D_{1200} = 0$ in every row.

Execution / CA rule	Median L (ms)	Median Ω (ms)
Python / real	99.569	99.627049
Python / byte floor	92.369	92.435552
SCP / byte floor	92.369	92.435552
Python / remainder	99.569	99.627049
SCP / remainder	99.569	99.627049

to 92.369 ms; per-ACK byte truncation still differs from the real-window model.

The current AIMD CA update retains the division remainder b_k:

$$A_k = MN_k + b_{k-1}, \quad I_k = \lfloor A_k / C_{k-1} \rfloor, \\ b_k = A_k - I_k C_{k-1}, \quad C_k = C_{k-1} + I_k. \tag{36}$$

The remainder starts at zero and resets on loss or timeout. An ACK may add zero bytes; forcing a one-byte minimum would defeat accumulation at large windows. SS retains its acknowledged-byte increase.

Three new native 1 GiB transfers and nine Python transfers (three per CA rule) complete successfully. The native runs retain exactly 100 ms feedback, matching checksums, no retransmission and identical event traces. The native binary SHA-256 begins `16ced6259764`. An independent ACK-index checker passes all nine Python runs. Table 4 and Figure 6 use complete, non-terminal groups 900–1200; the native and Python byte-rule controls also agree in actual span and accumulated start delay throughout groups 300–1200. Native short-tail and model rounded-up packets are excluded.

The actual native remainder median is 99.6270488517 ms for occupied span; actual span is 99.569 ms, with zero observed D_n in both selected ranges. At matching full-MSS ACK counts, the greatest absolute byte-window error against the real recurrence is 1.006703 bytes (ACK 120), and the error at ACK 750868 is −0.293415 bytes. Thus a strict "less than one byte" claim is false even though $b_k < C_{k-1}$: subsequent real and integer windows use different denominators, so their feedback recurrences differ. This ablation identifies discarded CA growth, alongside coarse dispatch, as the cause of the earlier mismatch; it does not change the gain.

9.4 Historical controls and the integer-window effect

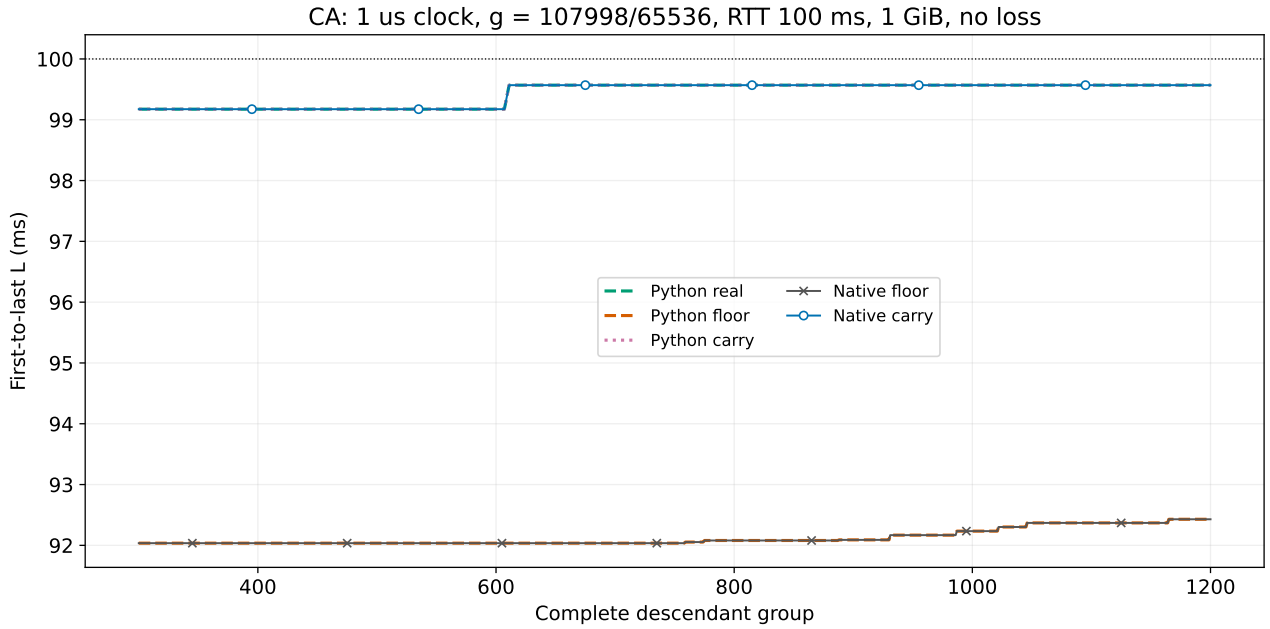
Policy and historical controls. The current controllers use (33); the research harness overrides gain for scans. The frozen 24 October CA policy control used the earlier $1 + M/(2C)$ policy: its three transfers completed at 1341.39 RTTs, while the corresponding real-window Python control completed at 1234.19889 RTTs. Those observations are retained as historical controls, not relabeled as runs of the new constant. Integer-byte CA growth is a separate and much larger difference between the two layers.

That earlier adaptive control also gives a concrete early witness: group 5 sends from 500 to 586 ms after the initial packet, for an actual span of 86 ms. Its reconstructed trailing slot gives an occupied span of 104.113873 ms; group 6 begins at 604 ms, four milliseconds beyond its nominal start. This is a native observation separate from the exact real-valued example in (11). Writing physical group starts and trailing slots as H_n and U_n, integer dispatch follows $H_{n+1} = \max(H_n + R, \lfloor U_n \rfloor_{\text{ms}})$. Thus sub-millisecond slot overruns need not produce an extra millisecond of drift.

The historical native CA campaign uses Table 1's conditions but sets both initial cwnd and ssthresh to M. This artificial initial state isolates CA; it is not ordinary TCP startup. The research target changes threshold initialization while reusing the shared core, virtual network, and pacing rules. For an ACK covering N_k new bytes, the former core used

$$C_k = C_{k-1} + \max\left(1, \left\lfloor \frac{MN_k}{C_{k-1}} \right\rfloor\right), \qquad C_0 = M. \tag{37}$$

This differs from the real-valued recurrence, even for full-MSS ACKs. RFC 5681 also discusses the integer-zero increment and



Coincident traces use different line styles and markers; all five conditions use the same Q16 gain.

Figure 6: Controlled CA comparison with effective gain 107998/65536 and 1 μs ticks. Actual SCP and Python agree when using the same byte-update rule; retaining the remainder restores the real-model late span. The figure shows L; Table 4 separately reports occupied span and start delay. Three repetitions per new configuration agree; no time or amplitude fitting is used.

rounding it to one byte; its equation (3) is one allowed CA method, not every implementation of Reno. For full packets, once $C > M^2/2 = 1{,}022{,}450$ bytes, (37) adds exactly one byte per ACK. Discarded fractional increments accumulate into a measurable difference.

In that frozen campaign, all 66 fixed-gain CA transfers and three adaptive-policy transfers completed with matching checksums, exactly 100 ms feedback, no retransmission, and no cap contact. Every observed ACK update matched (37) with zero byte error. At full-packet ACK $K = 750868$, the actual window was 1,476,947 bytes, versus 1,752,403.2934 bytes from (21): measured minus reference, divided by reference, is −15.719%. The final 584-byte ACK then raises the actual window to 1,476,948 bytes and is excluded from this full-packet comparison.

At the same full-packet ACK count, the verified integer window sequence is identical across gains; gain changes when those ACKs occur. Among Figure 7's fixed gains, the largest RTT-boundary difference from gain 1 is 2.340 MSS. From $g = 1$ to 8, completion improves by only 0.204%, while dispersion changes substantially. Table 5 reports the full span range over 1101 intervals $n = 100, \ldots, 1200$, rather than assuming exact periodicity. These millisecond observations do not make $2 \ln 2$ a CA transition. At matching elapsed times, native windows differ from the independent real-window models by up to about 21.9%; this includes different ACK timing as well as integer updates, unlike the matching-ACK-count comparison above.

The gain-difference sawtooth in Figure 7(b) follows the integer thresholds: the faster flow reaches a lower per-ACK increment first.

Table 5: Historical SCP CA completion and within-run span ranges, $n = 100, \ldots, 1200$.

Target gain	Last ACK (t/R)	Span range (ms)
1	1341.81	99
g_c	1339.48	99
2	1339.31	71–72
4	1339.14	33–34
8	1339.07	16

From RTT 814 to 815, gain 1 receives 713 ACKs at 2 bytes each, while gain 8 receives 715 at 1 byte each. Their window difference shrinks by $1426 - 715 = 711$ bytes, or 0.497203 MSS, exactly matching the plotted drop. Real-valued CA has no such integer step.

10 Interpretation and Reproducibility

What the constants explain. SS admits two packets per ACK, and the discrete sum over windows $W \to 2W$ gives $2 \ln 2$. CA usually admits one replacement; its extra whole-packet birth moves through growing groups $q \to 3q$, yielding front work $3 \ln 3/2$. This is the basis for different fixed gains in the two phases. The continuous $1 + M/(2C)$ result remains useful for explaining an earlier approach, rather than the deployed CA policy.

What the data show. Beyond the near-RTT spreading region, larger gain chiefly compresses groups. The SS sample $g_c - 0.05$ has multiplier 1.950624, while the critical target has multiplier two at the selected boundaries. CA windows at the same full-packet ACK

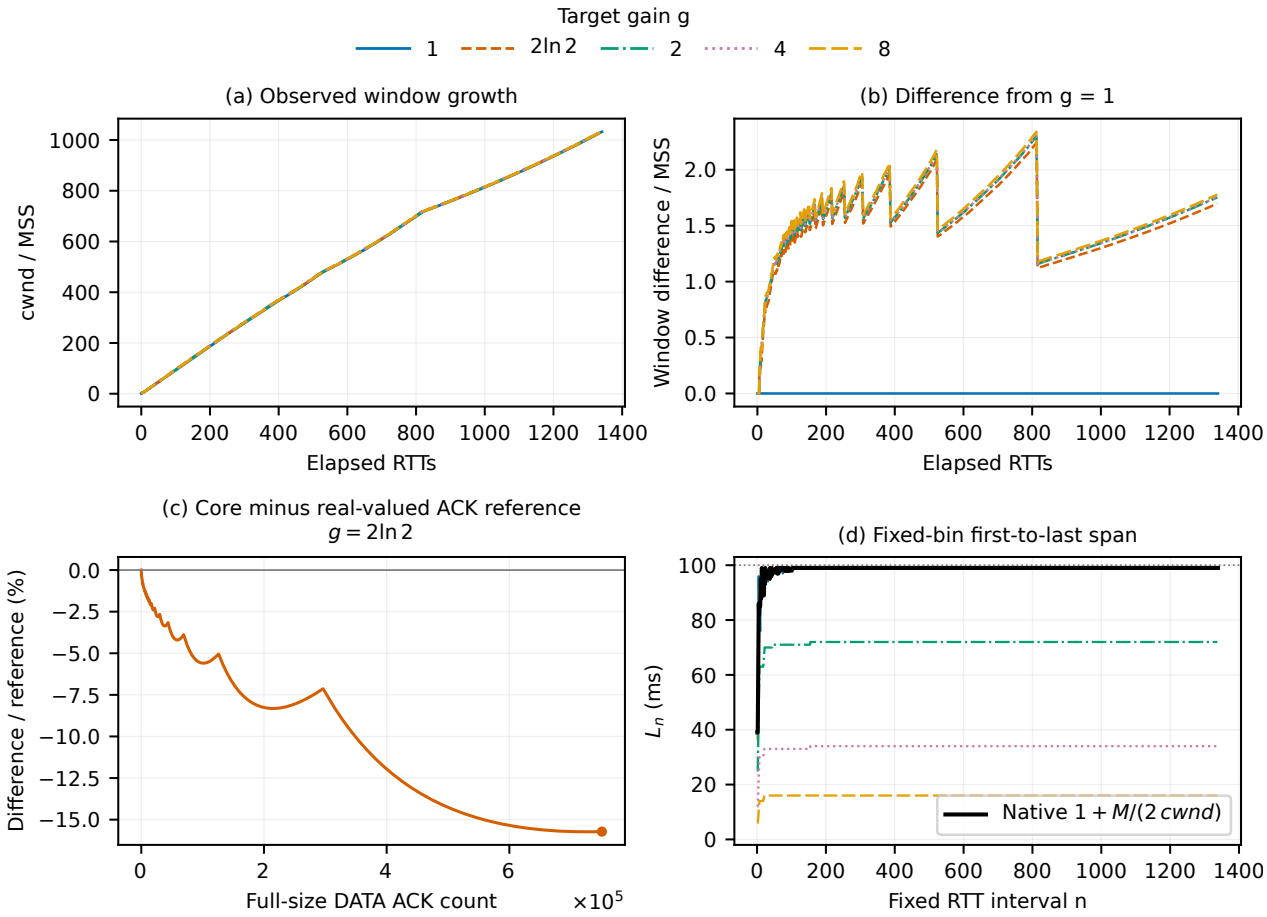


Figure 7: Frozen 24 October SCP CA campaign from one MSS, five representative fixed gains. (a) Observed RTT-boundary windows. (b) Differences from gain 1; sawtooth drops occur when a faster flow reaches a lower integer per-ACK increment first. (c) The $2\ln 2$ sample's window error against the real-valued recurrence at matching ACK counts, including the last full-packet ACK. (d) Actual fixed-interval spans, with the historical adaptive policy in black; final-send intervals are excluded. Lines connect observations without fitting.

count are identical across gains; gain changes the ACK times. The historical per-ACK byte truncation accounts for the much larger Python–SCP window deficit. Retaining the remainder and matching 1 μs clocks restores the late 99.569 ms span in actual SCP. Historical fixed-bin spans of 99 ms also reflect the 1 ms grid; complete groups and tail slots supply the cross-generation check.

Scope. The study starts one flow from one packet, with fixed feedback and separate loss-free phases. It excludes serialization, cross traffic, delayed ACKs and receiver pressure. These engineering gains do not establish Internet-wide fairness or remove finite-clock and integer-arithmetic effects. The derivation, reviewed Python clock model and actual SCP core have different evidentiary roles. The CA value is an engineering constant supported by the birth-front main term and gain sweeps; every finite group need not equal one RTT, and an exact all-time threshold for all packet histories is not asserted.

Relation to prior work. Reno's update is specified in RFC 5681 [1]. Pacing and TCP dynamics have long been studied [3]. RFC 9002 gives a cwnd/RTT pacing example [4]; the ICCRG pacing draft discusses compensation for SS growth [5]. Our contribution connects the packet construction and SS/CA sandwich explanations with reviewable execution of an integer protocol core.

Reproduction. The separate SS and CA branches provide compact tables, scripts and rerun commands. Large logs remain in Actions. The frozen 24 October campaigns used shared baseline `e41ac88`: SS at `631ab7b`, and CA at `d53c6d5` in Actions 40084981319. All 138 native runs passed the execution audit. The earlier independent Python rebuild comprised 264 runs, including 141 full-size requests; all passed the separate ACK-index comparison. The additional 22-gain CA engineering sweep is identified in Section 9. The common `test/models/` scripts provide the tick model, independent checker, native group reconstruction and plots. Actual protocol runs include endpoint checksum,

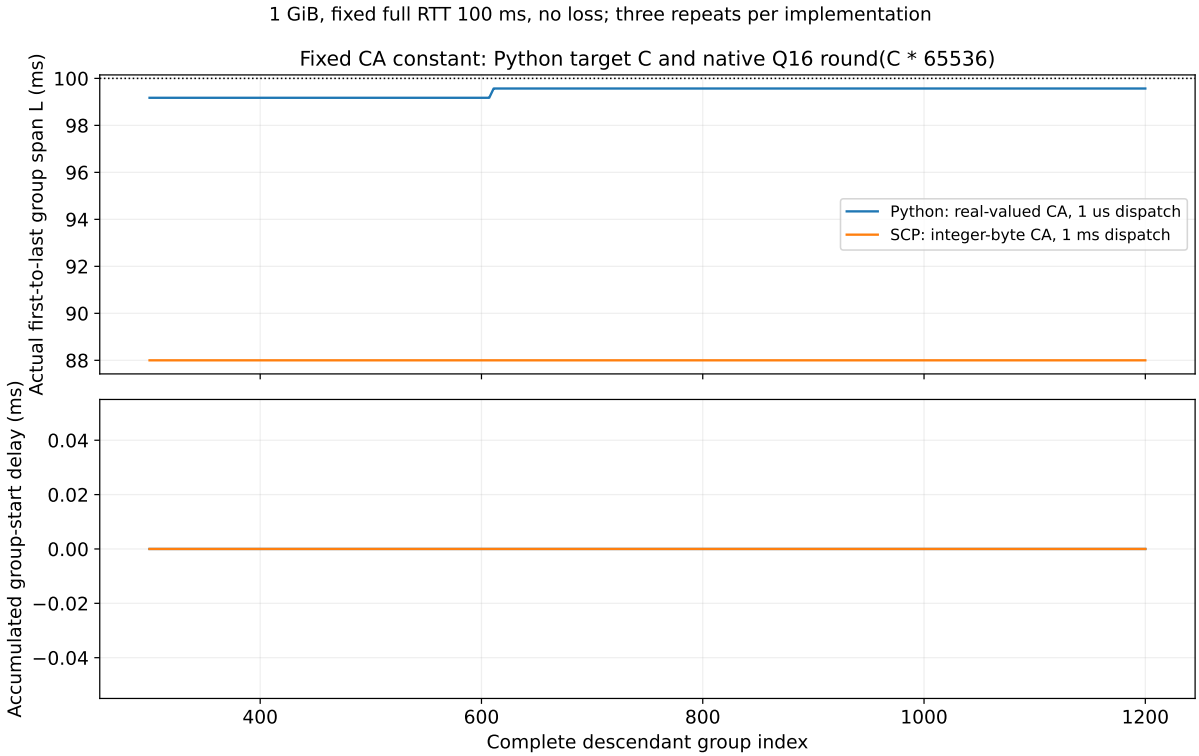


Figure 8: Historical fixed CA comparison before the clock and remainder corrections. Complete non-terminal descendant groups compare actual span, retained occupied span and accumulated start delay. Python uses real windows and $3\ln 3/2$; SCP uses per-ACK byte truncation, Q16 107998 and millisecond dispatch. Late spans are 99.569 and 88 ms respectively. All three repetitions in each layer agree; no time shift or amplitude fit is applied.

feedback, ACK-update and reservation audits. The historical local fixed-constant campaign is recorded in `data/ca-native-constant.csv`, `data/ca-native-transfers.csv` and the corresponding JSON source manifest; it is distinct from the frozen Actions campaigns above. The 26 October ablation adds only compact `data/ca-byte-ablation-late.csv`, `data/ca-byte-ablation-errors.csv`, `data/ca-byte-ablation-transfers.csv` and `data/ca-byte-ablation.json`, plus one comparison figure. Raw event logs remain outside the repository; the research branch supplies rerun commands. Its native transfers use an 8 MiB application backlog to keep data available without exhausting the send buffer; total payload is still exactly 1 GiB.

11 Conclusion

The useful route was to start with a packet and its returning ACK, derive continuous window references, return to whole-packet credit, and squeeze the relevant gap sum. SS gives $2\ln 2$ through windows $W \to 2W$. CA adds a birth front moving through $q \to 3q$, motivating the fixed engineering value $3\ln 3/2$. Python sweeps expose a near-RTT plateau and compression at greater gain, while actual SCP execution explains Q16, clock and integer-window differences. Keeping the construction, the earlier adaptive reference and the experimental corrections together makes the implementation choice and its limits understandable.

References

[1] M. Allman, V. Paxson, and E. Blanton. *TCP Congestion Control*. RFC 5681, 2009. https://www.rfc-editor.org/rfc/rfc5681.

[2] Linux v6.12, `net/ipv4/tcp_output.c`, timestamp assignment and post-send pacing update. Source at commit `adc218676eef`.

[3] A. Aggarwal, S. Savage, and T. Anderson. Understanding the performance of TCP pacing. *IEEE INFOCOM*, 2000, pp. 1157–1165.

[4] J. Iyengar and I. Swett. *QUIC Loss Detection and Congestion Control*. RFC 9002, Section 7.7, 2021. https://www.rfc-editor.org/rfc/rfc9002.

[5] M. Welzl et al. *Pacing in Transport Protocols*. Internet-Draft, draft-irtf-iccrg-pacing-02, 2026. Work in progress. https://datatracker.ietf.org/doc/html/draft-irtf-iccrg-pacing-02.

A Checking the Exact SS Queue

A.1 Checking the SS Reservation Clock

Write $c = 2/g$, $H_k = \sum_{i=1}^{k} 1/i$, and $D(q) = H_{2q} - H_q$. Group ACK indices as $q \le k < 2q$, where $q = 2^m$, and put $B_m = b_{2^m}$. These are analytical groups, not batches processed by the implementation. The exact solution is

$$B_{m+1} = B_m + \max\{1, cD(2^m)\}, \quad B_0 = 1, \tag{38}$$

$$b_k = B_m + c(H_k - H_{2^m}), \quad 2^m \le k < 2^{m+1}. \tag{39}$$

To verify it, note that $b_k = \max(a_k, b_{k-1} + c/k)$ for $k \ge 2$, and $a_{2j} = b_j + 1$, $a_{2j+1} = b_j + 1 + c/[2(j+1)]$. At the first index q of a group, the two candidate times are $B_{m-1} + 1$ and $B_{m-1} + cD(q/2)$,

proving (38). Set $\Delta = B_m - B_{m-1} \geq 1$. At interior even indices,

$$T_{2j-1} - a_{2j} = \Delta - 1 + c[D(j) - D(q/2)] > 0. \tag{40}$$

At interior odd indices the bracket additionally contains $1/(2j+1) - 1/(2j+2) > 0$. Since $D(j+1) - D(j) = 1/[(2j+1)(2j+2)] > 0$, the queue branch wins throughout each group's interior. Summing its increments proves (39) by induction. In particular,

$$b_k = cH_k + 1 - c + \sum_{i=0}^{\lfloor \log_2 k \rfloor - 1} \left[1 - cD(2^i)\right]_+. \tag{41}$$

Here $[x]_+ = \max(x, 0)$. Together with (14) and (12), this determines the window at every time without fitting a growth curve.

A.2 Checking the SS Tail Identity

We return to the construction that motivated the model. Analytical group m, with $q = 2^m$, contains packets $2q-1, \ldots, 4q-2$ and corresponds to descendant generation $m+1$. Write its normalized tail time as $E_m = s_{4q-2}$ and its span as $\ell_m = E_m - B_m = L_{m+1}/R$. For $m \geq 1$, the preceding group's tail $E_{m-1} = s_{2q-2}$ produces ACK $2q-1$, which permits this group's final pair. Put

$$\begin{aligned} d_m &= b_{2q-1} - (E_{m-1} + 1), \\ \delta_m &= \frac{1}{2gq}, \qquad e_m = B_m - B_{m-1} - 1. \end{aligned} \tag{42}$$

Here $d_m \geq 0$ is the old-queue wait after that ACK, δ_m is the final pair's spacing, and e_m is the group start's extra drift. The tail moves by one RTT, the wait, and that spacing. Subtracting the group start gives the exact accounting identity

$$\ell_m = \ell_{m-1} + d_m + \delta_m - e_m. \tag{43}$$

Thus following the tail does reduce the problem to accumulated increments, but the pair spacing alone is not the whole increment. Substituting (39) into $d_m - e_m$ cancels the endpoint corrections and gives

$$d_m - e_m = \frac{2}{g}[D(q) - D(q/2)]. \tag{44}$$

With $\ell_0 = 1/(2g)$, the increments telescope. Here analytical group $m = 0$ is descendant generation 1, so $\ell_m = L_{m+1}/R$. Summing the pair spacings and the waiting-minus-drift terms gives the finite expression in (15). This also verifies that within each group the queue branch supplies a continuous reservation train; it was not assumed from the number of admitted packets alone.

B Exact CA Packet Indices and Tail Accounting

These indices implement parent-ACK groups without using time bins. Let $\eta_k = [T_{k-1} - a_k]_+$ be the queue wait after ACK k. Its final new packet leaves at

$$z_k = a_k + \eta_k + \frac{m_k - 1}{g u_k}. \tag{45}$$

Number packets from zero. ACK k corresponds to packet $k-1$; its last child has index $J_k = k + \lfloor u_k \rfloor - 1$, with $J_0 = p_0 - 1$. Parent-ACK groups obey

$$\begin{aligned} \alpha_0 &= 0, \qquad \beta_0 = p_0 - 1, \\ \alpha_{n+1} &= \beta_n + 1, \qquad \beta_{n+1} = J_{\beta_n + 1}. \end{aligned} \tag{46}$$

Let $h_n = s_{\alpha_n}$, $t_n = s_{\beta_n}$, $\ell_n = t_n - h_n = L_n/R$, and $k_n = \beta_n + 1$. With

$$d_n = [T_{k_n - 1} - (t_n + 1)]_+, \quad e_n = h_{n+1} - h_n - 1,$$

the last-child construction gives the exact identities

$$\begin{aligned} t_{n+1} &= t_n + 1 + d_n + \frac{m_{k_n} - 1}{g u_{k_n}}, \\ \ell_{n+1} &= \ell_n + d_n + \frac{m_{k_n} - 1}{g u_{k_n}} - e_n. \end{aligned} \tag{47}$$

Since $\beta_n = J_{\alpha_n}$, the old tail slot is $T_{\alpha_n} = t_n + 1/(g u_{\alpha_n})$. Thus

$$e_n = \left[\ell_n + \frac{1}{g u_{\alpha_n}} - 1\right]_+. \tag{48}$$

C The CA Actual-Span Bound

This proof applies to exact reservation times, real-valued CA, fixed gain, full MSS packets and the one-packet initial state. Packet groups are those defined in Appendix B. Consider an old group of $p = \lfloor u \rfloor$ packets, where u is the window just before its first ACK is processed. In normalized time, write its sends as $s_1, \ldots, s_p$ and set $\theta = 1/(gu)$. Their original reservation windows were no greater than u, so $s_{j+1} - s_j \geq \theta$ and their retained trailing slot is $s_p + \theta$. The next p ACKs permit $m_j = 1 + \epsilon_j$ new packets with $\sum \epsilon_j \leq 1$: their total window increase is at most $p/u \leq 1$. Let u_j denote the local window after this group's ACK j, with $u_0 = u$. Each new spacing $\delta_j = 1/(g u_j)$ is strictly below θ.

The new group's first send is $B = \max(s_1 + 1, s_p + \theta)$. Put $P_j = \sum_{i=j}^{p} m_i \delta_i$. Expanding the reservation maximum gives its exact span:

$$\begin{aligned} \ell_{\text{new}} = \max\{&P_1 - \delta_p, \\ &\max_{j \geq 2}(s_j + 1 - B + P_j - \delta_p)\}. \end{aligned} \tag{49}$$

The service branch contains at most p spacings, hence $P_1 - \delta_p < p\theta \leq 1/g$. Each arrival branch satisfies

$$P_j - \delta_p < (p - j + 1)\theta \leq s_p - s_j + \theta \leq B - s_j,$$

so it is below one. The initial single-packet case is immediate. Thus

$$L_n < R \max(1, 1/g), \qquad g \geq 1 \Rightarrow L_n < R. \tag{50}$$

Conversely, after these ACKs $u_p \leq u + p/u \leq u + 1 < p + 2$, and the new group has at least p packets. Therefore

$$L_{\text{new}} > \frac{(p-1)R}{g(p+2)}. \tag{51}$$

With unlimited data the group-start ACK index is unbounded; (21) then implies $p \to \infty$. Consequently $L_n \to R$ at $g = 1$ and $L_n \to R/g$ for $0 < g < 1$. If the objective is only that every complete group's actual span remain at most R, its fixed-gain threshold is one in this ideal model. This is not a solution of the window-utilization objective.

D Packet-Time Comparison and the Burst Limit

Both systems use fixed feedback and one-MSS growth per ACK. The comparison concerns ideal packet times; integer execution requires its own check.

D.1 The Fixed-Batch Comparator Respects Flight

Let the preceding and current batch sizes be $p = F_{n+1}$ and $q = F_{n+2}$ in round $n \geq 1$. After phase $0 \leq u < 1$, its flight is

$$F(n+u) = p + \lfloor qu \rfloor - \lfloor pu \rfloor \leq q. \tag{52}$$

Its current window is at least q, so the comparator respects the actual-flight limit. It does not reserve all future flight positions at planning time: it can rely on ACKs arriving during the batch. This distinguishes its planning rule from the reference scheduler's invariant $F + Q = w$.

D.2 Every Reference Packet Leaves No Later at Unit Gain

At $g = 1$, $2D(q) \geq 1$ for all $q \geq 1$. Equation (41) reduces to $b_k = 2H_k - 1$, giving

$$s_0 = 0, \qquad s_{2k-1} = 2H_k - 1, \qquad s_{2k} = 2H_k - 1 + \frac{1}{k+1}. \tag{53}$$

The consecutive spacings decrease with packet index:

$$s_{j+1} - s_j = \frac{1}{\lfloor (j+1)/2 \rfloor + 1}, \qquad j \geq 0. \tag{54}$$

The fixed batch of size q starts with packet index $J_n = p + q - 2$. Its timestamps are

$$\widehat{s}_j = n + \frac{j - J_n}{q}, \qquad J_n \leq j < J_n + q. \tag{55}$$

Within this batch, the successive differences of $s_j - \widehat{s}_j$ are the decreasing spacings above minus $1/q$. A maximum is attained at $j_* = 2q-3$. This index belongs to the batch because $j_* - J_n = q-p-1 \geq 0$ and $J_n + q - 1 - j_* = p \geq 1$. Substitution yields

$$\max_{J_n \leq j < J_n + q} (s_j - \widehat{s}_j) = -E_n, \qquad E_n = n + 2 - \frac{p-1}{q} - 2H_q. \tag{56}$$

Initially $p = 1, q = 2$, and $E_1 = 0$. In the next batch, (p, q) becomes $(q, p+q)$, so

$$\begin{aligned} E_{n+1} - E_n &= 1 - \frac{q-1}{p+q} + \frac{p-1}{q} - 2\sum_{i=1}^{p} \frac{1}{q+i} \\ &= 2\sum_{i=1}^{p} \frac{i(p-i)}{q(q+p)(q+i)} \geq 0. \end{aligned} \tag{57}$$

For the second equality, subtract $1/(q+i)$ from $(q+p-i)/[q(q+p)]$; the difference is $i(p-i)/[q(q+p)(q+i)]$. Summing these identities gives the second line of (57). Thus $E_n \geq 0$, proving $s_j \leq \widehat{s}_j$ in every batch. Packet zero is common to both systems. Earlier corresponding sends give earlier corresponding ACKs, hence no smaller window at any observation time.

D.3 Extension to Larger Gains

For $g_2 \geq g_1 \geq 1$, induction over ACK batches gives $s_j(g_2) \leq s_j(g_1)$. The initial send is common. If earlier packet times are ordered, their fixed-delay ACK times are ordered. The previous reservation tail is also ordered, since every newly added spacing decreases as gain increases. Taking the maximum of the ACK time and the previous tail preserves the order, and so does adding the second packet's spacing.

Combining this monotonicity with the unit-gain comparison and the flight upper bound proves, specifically for (14),

$$MF_{n+2} \leq \mathrm{cwnd}_g(nR) \leq M2^n, \qquad g \geq 1. \tag{58}$$

The actual-time comparison is essential. Merely appending more packets earlier does not order their departures under arbitrary queue or timestamp rules.

D.4 Approaching Burst Sending

For $g \geq g_c$, $B_m = m + 1$. For any fixed packet index j in group m, its timestamp therefore has the form $s_j = m + 1 + d_j/g$, with $d_j \geq 0$ independent of gain. Including packet zero,

$$\lim_{g \to \infty} s_j(g) = \lfloor \log_2(j+1) \rfloor. \tag{59}$$

This is the immediate-send packet sequence. At finite high gain, $W_n = 1 + 2^{n-1}$, whereas immediate sending gives 2^n: ACKs just after a boundary coalesce onto it as gain tends to infinity. Convergence of packet times therefore does not imply pointwise window convergence at an event boundary.